ДИСЦИПЛИНА **Математическая логика и теория алгоритмов**
(полное наименование дисциплины без сокращений)

ИНСТИТУТ **информационных технологий**

КАФЕДРА **прикладной математики**
(полное наименование кафедры)

ВИД УЧЕБНОГО МАТЕРИАЛА **Лекция**
(в соответствии с пп.1-11)

ПРЕПОДАВАТЕЛЬ **Батенков Кирилл Александрович**
(фамилия, имя, отчество)

СЕМЕСТР **1, 2024-2025**
(указать семестр обучения, учебный год)

1. ВИДЫ АЛГОРИТМОВ

1.1. Алгоритмы сортировки, поиска

Многие программы и алгоритмы работают с массивами данных. Массив группирует данные одною и того же типа в единую сущность. Вы можете думать о массиве как о таблице, которая при наличии индекса некоторой записи позволяет получить элемент массива с указанным индексом.

Индексы в массиве представляют собой последовательные числа, которые могут начинаться с любого числа, но обычно индексы начинаются с 1. Имея имя массива и индекс, мы можем объединить их с помощью квадратных скобок для указания конкретного элемента массива. Например, i-й элемент массива A обозначается как $A[i]$.

Массивы в компьютере обладают одним важным свойством: обращение к любому элементу массива выполняется за одно и то же время. Когда вы передаете компьютеру индекс i в массиве, он обращается к i-му элементу так же быстро, как и к первому, независимо от значения i.

Давайте рассмотрим алгоритм: поиск определенного значения в массиве. То есть мы имеем массив и хотим узнать, какая запись в массиве (если таковая имеется) имеет определенное значение. Чтобы понять, как можно организовать поиск в массиве, давайте представим массив как длинную полку, полную книг, и предположим, что нам надо узнать, где на полке находится книга Шолохова. Книги на полке могут быть каким-то образом упорядочены, например в алфавитном порядке по фамилии автора или в алфавитном порядке по названию, или по некоторому внутреннему библиотечному номеру. А может быть, они стоят так, как на моей полке у меня дома, где в силу моей неорганизованности книги стоят как попало.

Если вы не можете заранее быть уверены, что книги стоят на полке в некотором порядке, как бы вы искали книгу Шолохова? Вот алгоритм, которому следовал бы я. Я бы начал елевою конца полки и посмотрел на первую книгу. Если это Шолохов, я обнаружил искомую книгу. В противном случае я бы посмотрел на следующую книгу справа, и если эта книга Шолохова, то я обнаружил искомую книгу. Если же нет, то я бы продолжил идти вправо, просматривая книгу за книгой, пока не нашел бы книгу Шолохова или пока не натолкнулся бы на правую стенку полки (и сделал бы вывод, что на этой полке книги Шолохова нет).

Вот как мы можем описать задачу поиска в вычислительной терминологии. Будем рассматривать полку как массив книг. Крайняя слева книга находится в позиции 1, следующая книга справа он нее находится в позиции 2, и т.д. Если на полке у нас есть п книг, то крайняя справа книга находится в позиции п. Мы хотим найти номер позиции любой книги Шолохова на полке.

В качестве обобщенной вычислительной задачи мы получаем массив Л (вся книжная полка, на которой мы ищем интересующую нас книгу) с л элементами

(отдельными книгами), и при этом надо выяснить, присутствует ли в массиве A значение x (книга Шолохова) Если да, то мы хотим знать индекс i, такой, что $A[i] = x$ (т.е. в i-й позиции на полке стоит книга Шолохова). Мы также должны иметь возможность каким-то образом сообщить, что массив A не содержит элемент x (на полке нет книг Шолохова). Мы не ограничиваем себя предположением, что x содержится в массиве не более одного раза (возможно, на полке несколько книг Шолохова), так что если элемент x присутствует в массиве A, он может встретиться там несколько раз. Все, что мы хотим от алгоритма поиска, – произвольный индекс, по которому мы найдем элемент x в массиве A. Мы предполагаем, что индексы массива начинаются с 1, так что его элементами являются элементы с $A[1]$ по $A[n]$.

Если мы ищем книгу Шолохова, начиная с левою конца полки и проверяя поочередно все книги слева направо, такой метод называется *линейным поискам*. В терминах массива в памяти компьютера мы начинаем с начала массива (первого его элемента), поочередно проверяя все его элементы ($A[1]$, затем $A[2]$, затем $A[3]$ и так далее до $A[n]$) и записывая место, где мы находим x (если мы вообще находим его).

Приведенная далее процедура Linear-Search получает три параметра, которые в спецификации разделены запятыми.

Процедура $Linear - Search(A, n, x)$

Вход:

A: массив.

n: количество элементов массива A, среди которых выполняется поиск.

x: искомое значение.

Выход: либо индекс i, для которого $A[i] = x$, либо специальное значение not-found, которое может представлять собой любой некорректный индекс массива, такой как 0 или произвольное отрицательное значение.

1. Установить значение answer равным not-found.

2. Для каждого индекса i, пробегающего поочередно значения от 1 до n:

   A. Если $A[i] = x$, установить значение answer равным i.

3. В качестве выходного вернуть значение answer.

В дополнение к параметрам A, n, x процедура Linear-Search использует переменную с именем answer. Процедура присваивает начальное значение not-found переменной answer на шаге 1. Шаг 2 проверяет каждую запись массива от $A[1]$ до $A[n]$, не содержит ли она значение x. Всякий раз, когда запись $A[i]$ равна x, шаг 2A присваивает переменной answer текущее значение индекса i. Если x присутствует в массиве, то выходное значение, возвращаемое на шаге 3, представляет собой последний индекс, где встретился элемент x. Если же элемент x в массиве отсутствует, то проверка равенства на шаге 2A никогда не будет истинной, и процедура вернет значение not-found, присвоенное переменной answer на шаге 1.

Прежде чем мы продолжим обсуждение линейного поиска, скажем несколько слов о том, как описать неоднократные действия, такие как на шаге 2. Такие

повторяющиеся действия довольно часто встречаются в алгоритмах, например, для выполнения некоторых действий для переменной, принимающей значения из определенного диапазона. Когда мы выполняем такие неоднократные действия, это называется циклам, а каждое однократное выполнение действий – итерацией цикла. Описывая цикл на шаге 2, я написал «для каждого индекса», пробегающего поочередно значения от 1 до n». Далее вместо такого словесного описания я буду использовать запись «для $i = 1$ до n», которая короче, но передает ту же структуру. Обратите внимание, что, когда я записываю цикл таким образом, переменная цикла (в данном случае – i) получает начальное значение (в данном случае – i), и в каждой итерации цикла значение переменной цикла сравнивается с пределом (в данном случае – n). Если текущее значение переменной цикла не превышает предел, мы делаем все, что указано в теле цикла (в данном случае это строка 2A). После выполнения итерации тела цикла выполняется инкремент переменной цикла – т.е. прибавление к ней 1. – после чего выполняется возврат к заголовку цикла, где новое значение переменной цикла сравнивается с пределом. Проверка значения переменной цикла, выполнение его тела и увеличение значения переменной выполняется многократно, до тех пор, пока значение переменной цикла не превысит предел. Затем выполнение продолжается с шага, следующего после тела цикла (в данном случае с шага 3). Цикл вида «для $i = 1$ до n» выполняет n итераций и $n + 1$ проверку на превышение значения предела (поскольку значение переменной цикла превысит предел при $(n + 1)$-й проверке).

Я надеюсь, для вас очевидно, что процедура Linear-Search всегда возвращает правильный ответ. Но вы могли заметить, что эта процедура неэффективна: она продолжает поиск в массиве даже после того, как индекс i, для которого $A[i] = x$, уже найден. Вряд ли вы продолжаете поиск книги на полке после того, как уже нашли ее. Так что мы можем спроектировать нашу процедуру линейного поиска так, чтобы поиск прекращался, как только он находит в массиве значение x. Далее мы считаем, что команда возврата значения немедленно прерывает выполнение процедуры и возвращает указанное значение вызывающему коду.

Процедура $Better - Linear - Search(A, n, x)$

Вход и выход: те же, что и в Linear-search.

1. Для $i = 1$ до n :

A. Если $A[i] = x$, вернуть значение i в качестве выхода процедуры.

Вернуть в качестве выходного значение Not-found.

Поверите ли вы в это или нет, но линейный поиск можно сделать еще более эффективным. Заметим, что при каждом выполнении итерации цикла из шага 1 процедура Better-Linear-Search выполняет две проверки: проверка в первой строке для выяснения, не вышло ли значение переменной цикла за допустимые рамки (т.е. выполняется ли неравенство $i \leq n$), и если не вышло, то выполняются очередная итерация цикла и проверка равенства во второй строке. В терминах поиска на

книжной полке эти проверки выглядят так, как будто бы для каждой книги вы проверяли две вещи - добрались ли вы до конца полки и не является ли автором следующей книги Шолохов? В случае с полкой особых неприятностей от того, что вы пройдете немного дальше полки, не будет (конечно, если только вы не ударитесь о стену), но в случае компьютерной программы обращение к элементам массива за его концом обычно заканчивается очень плохо. Например, это может привести к повреждению данных или аварийному завершению программы.

Можно исправить алгоритм так, что вам придется выполнять для каждой книги только одну проверку. Что если бы вы точно знали, что на полке есть книга Шолохова? Тогда вы были бы твердо уверены, что найдете ее, так что до конца полки вы никогда не доберетесь. Достаточно просто проверять все книги по очереди, пока не встретитесь с книгой Шолохова.

Но представим, что вы ищете книгу на чужой полке или на своей, но не помните, на какую именно полку засунули книги Шолохова - словом, вы не уверены, что на полке есть требуемая книга. Вот что вы можете сделать в этом случае. Возьмите пустую коробку размером с книгу и на узкой ее стороне (имитирующей корешок книги) напишите «Шолохов. Поднятая целина». Замените крайнюю справа книгу этой коробкой, после чего при поиске вдоль полки слева направо вам нужно будет проверять только автора книги, но не то, последняя ли эта книга на полке, потому что теперь вы точно знаете, что найдете что-то, на чем написано «Шолохов». Единственный вопрос заключается в том, нашли ли вы настоящую книгу или ее заменитель. Если это заменитель, значит, книги Шолохова на полке нет. Но такую достаточно простую проверку надо выполнить только один раз, в конце поиска, а не для каждой книги на полке.

Имеется еще одна деталь, о которой надо упомянуть: что если единственная книга Шолохова на вашей полке была крайней справа? Если вы заменили ее пустой коробкой, поиск завершится, когда вы ее найдете, и вы можете ошибочно заключить, что искомой книги на полке нет. Поэтому следует выполнить еще одну проверку - для выявления такой возможности. Но это тоже только одна проверка, не выполняемая для каждой книги на полке.

В терминах компьютерного алгоритма мы помещаем искомое значение x в последнюю позицию $A[n]$ после сохранения содержимого $A[n]$ в другой переменной. Когда мы нахідим x, мы выполняем проверку, действительно ли мы его нашли. Значение, которое мы помещаем в массив, называется одраничителем, но его можно рассматривать как пустой ящик.

Процедура $Sentinel - Linear - Search(A, n, x)$

Вход и выход: те же, что и в Linear-Search.

1. Сохранить $A[n]$ в переменной last и поместить x в $A[n]$.

2. Установить i равным 1 .

3. Пока $A[i] \neq x$, выполнять следующие действия:

А. Увеличить i на 1.

4. Восстановить $A[n]$ из переменной last.

5. Если $i < n$ или $A[n] = x$, вернуть значение i в качестве выхода процедуры.

6. В противном случае вернуть в качестве возвращаемого значения not-found.

Шаг 3 представляет собой цикл, но цикл, в котором не происходит увеличения переменной-счетчика. Вместо этого итерации цикла выполняются до тех пор, пока выполняется условие цикла; в данном случае это условие $A[i] \neq x$. Цикл работает следующим образом. Сначала выполняется проверка условия цикла (в данном случае $-A[i] \neq x$), и если оно истинно, то выполняется тело цикла (в данном случае - строка 3А, которая увеличивает переменную i на 1). Затем выполняется возврат в начало цикла, и, если условие цикла истинно, снова выполняется тело цикла. И так проверка условия с последующим выполнением тела цикла продолжается до тех пор, пока при очередном выполнении условие не окажется ложным. Тогда алгоритм продолжается с шага, следующего за телом цикла (в данном случае - со строки 4).

Процедура Sentinel-Linear-Search оказывается немного более сложной по сравнению с первыми двумя процедурами линейного поиска. Поскольку она помещает x в $A[n]$ на шаге 1 , мы гарантируем, что $A[i]$ будет равно x при некоторой проверке на шаге 3. Как только это произойдет, мы прекращаем выполнение цикла на шаге 3 , и индекс i больше изменяться не будет. Прежде чем мы сделаем что-либо еще, на шаге 4 мы восстанавливаем исходное значение $A[n]$. (Моя мать всегда учила меня, попользовавшись чем-то, положить все назад, откуда взял.) Затем мы должны определить, действительно ли мы нашли в массиве x. Поскольку мы поместили x в последний элемент, $A[n]$, мы знаем, что если мы нашли x в $A[i]$, где $i < n$, то мы действительно нашли x и можно смело возвращать индекс, хранящийся в переменной i. Но что делать, если мы нашли x в $A[n]$? Это означает, что мы не нашли x до $A[n]$, так что нам надо проверить, действительно ли $A[n]$ равен x. Если это так, то мы возвращаем индекс n, который в этом случае равен i, но если $A[n] \neq x$, то следует возвратить значение not-found. Эти проверки выполняются на шаге 5 , и, если x изначально находился в массиве, на этом же шаге возвращается корректное значение индекса. Если же x найден только потому, что это значение было помещено в массив на шаге 1 , то шаг 6 вернет значение not-found. Хотя процедура Sentinel-Linear-Search и выполняет две проверки по окончании цикла, на каждой итерации выполняется только одна проверка, так что эта процедура оказывается более эффективной, чем процедуры Linear-Search и Betier-Linear-Search.

Можно ли как-то улучшить имеющиеся алгоритмы? Ответ: это зависит от обстоятельств. Если мы ничего не знаем о порядке элементов в массиве, то ничего лучшего мы не добьемся. В худшем случае мы должны просмотреть все n элементов, потому что, если мы не смогли найти значение, которое мы ищем, среди первых $n - 1$ элементов, оно может оказаться в последнем, n-м элементе. Таким

образом, мы не можем достичь времени работы в наихудшем случае лучшего, чем $\Theta(n)$, если ничего неизвестно о порядке элементов в массиве.

Предположим, однако, что массив отсортирован в неубывающем порядке, те. каждый элемент массива меньше или равен элементу, следующему в массиве за ним, согласно некоторому определению отношения "меньше, чем". В этой главе мы увидим, что если массив отсортирован, то мы можем использовать простой метод, известный как бинарный поиск, чтобы найти нужный элемент в массиве из n элементов за время всего лишь $O(\lg n)$. Как мы видели в главе 1 , "Что такое алгоритмы и зачем они нужны", значение $\lg n$ растет с ростом n очень медленно, и поэтому бинарный поиск превосходит линейный в наихудшем случае.'

Что это означает – что один элемент меньше другого? Если элементы являются числами, это очевидно. Если элементы являются строками символов текста, можно говорить о лексикографическом порядке: один элемент меньше другого, если в словаре он должен располагаться первым. Когда элементы представляют собой другие виды данных, мы должны определить, что означает понятие «меньше». При наличии некоторого точного понятия "меньше, чем" можно определить, отсортирован ли массив.

Вспомним пример книжной полки, «Описание и оценка компьютерных алгоритмов». Мы могли бы рассортировать книги в алфавитном порядке по фамилии автора, в алфавитном порядке по названию или, если в библиотеке имеется каталог, по номерам в каталоге. Будем считать, что книги на полке отсортированы в алфавитном порядке по автору, слева направо. На полке может содержаться более одной книги одного автора - возможно, у вас несколько книг Уильяма Шекспира. Если требуется искать не просто книгу Шекспира, а конкретную его книгу, то будем считать, что если на полке стоят две книги одного и того же автора, то та из книг, название которой идет в алфавитном порядке первым, должна находиться левее. Может оказаться и так, что все, что нас интересует, - это имя автора, так что при поиске нас устроит любая книга Шекспира. Информация, которую мы сопоставляем с книгой при поиске, называется ключам. В нашем примере с книжной полкой ключом является только имя автора, а не сочетание сначала имени автора, а затем названия книги при наличии нескольких книг одного автора.

Каким же образом нам получить отсортированный массив? Существует четыре базовых алгоритма – сортировка выбором, сортировка вставкой, сортировка слиянием и быстрая сортировка, – предназначенными для сортировки массива, и применением каждого из этих алгоритмов к нашей книжной полке. Каждый алгоритм сортировки имеет свои достоинства и свои недостатки. Все алгоритмы сортировки, которые приведены в данной главе, имеют время работы в худшем случае либо $\Theta(n^2)$, либо $\Theta(n\ \lg n)$. Таким образом, если вы собираетесь выполнить лишь несколько поисков, то лучше остановиться на линейном поиске. Но если вы

собираетесь выполнять поиск много раз, то может иметь смысл сначала отсортировать массив, а затем применять бинарный поиск.

Сортировка – важная задача сама по себе, а не только как шаг предварительной обработки данных для бинарного поиска. Подумайте обо всех данных, которые должны быть отсортированы, - таких как записи в телефонной книге, чеки в ежемесячной выписке банка да и просто результаты работы поисковой веб-системы, отсортированные по уровню релевантности. Кроме того, сортировка часто является отдельным шагом в другом алгоритме. Например, в компьютерной графике объекты часто перекрывают один другой. Программа, которая отображает объекты на экране, должна отсортировать их в соответствии с отношением «находится выше», чтобы затем иметь возможность изображать их один за другим снизу вверх.

Прежде чем идти дальше, пару слов о том, что такое сортировка. В дополнение к ключу (который при сортировке мы будем называть ключам сортировки) сортируемые элементы обычно содержат некоторую информацию, которую мы называем сопутствующими данными. Сопутствующие данные представляют собой информацию, которая связана с ключом сортировки и перемещается при перемещении элементов вместе с ключом. В нашем примере с книжной полкой ключом сортировки является фамилия автора, а сопутствующими данными - сама книга.

Пояснить концепцию сопутствующих данных достаточно просто следующим образом, который заставляет быстро понять суть проблемы. Таблица успеваемости, отсортированная по столбцу фамилий студентов (ключом сортировки являются фамилии, а сопутствующими данными – успеваемость). Если теперь отсортировать таблицу так, что ключом сортировки будет успеваемость (в порядке ее убывания), а фамилии студентов – сопутствующими данными, и при этом не перемещать фамилии, оставив их в первом столбце и сортируя только столбец с данными об успеваемости...

Вот некоторые другие примеры ключей сортировки и сопутствующих данных. В телефонной книге ключом сортировки является фамилия и имя абонента, а сопутствующими данными - адрес и номер телефона. В поисковой системе ключ сортировки представляет собой меру релевантности, а сопутствующими данными является URL веб-страницы, а также любая иная информация о странице, которую хранит поисковая система.

При работе с массивами мы действуем так, как будто каждый элемент содержит только ключ сортировки. При реализации любого из приведенных здесь алгоритмов сортировки следует убедиться, что при перемещении ключа сортировки перемещаются и сопутствующие данные, или по крайней мере указатель на них.

Чтобы аналогия с книжной полкой была применима к компьютерным алгоритмам, необходимо, чтобы полка и книги обладали двумя не слишком реалистичными свойствами. Во-первых, все книги на полке должны иметь один и тот же размер, потому что в массиве в компьютере все элементы массива имеют

одинаковый размер. Во-вторых, все позиции книг на полке можно пронумеровать числами от 1 до n, и каждую такую позицию мы будем называть слотам. Слот 1 – крайний слева, а слот n - крайний справа. Как вы, наверное, догадались, каждому слоту на полке соответствует запись массива.

В обыденной речи сортировка может означать нечто, существенно отличное от того, что это слово означает в вычислительной технике. Словарь на моем компьютере определяет термин «сортировка» как «систематическая организация в группах; разделение в соответствии с типом, классом и т.д.». Так что этим термином, например, можно назвать раскладывание по полкам шкафа одежды: рубашки - в одно место, галстуки - в другое и т.д. В мире компьютерных алгоритмов сортировка означает размещение элементов в некотором строго определенном порядке, а «систематическая организация в группах» называется «группировкой».

Рассмотрим бинарный поиск, который требует, чтобы массив, в котором он выполняется, был отсортирован. Бинарный поиск имеет то преимущество, что для поиска в массиве из n элементов требуется время $O(\lg n)$.

В нашем примере с книжной полкой мы работаем с книгами, уже отсортированными по автору слева направо. Давайте, используя в качестве ключа фамилию автора книги, поищем любую книгу Маяковского. Конечно, буква "М" находится примерно в средине алфавита, но даже если бы мы искали книгу Шолохова, это еще не гарантировало бы, что она будет близко к правому концу полки, – в конце концов, на полке может оказаться большое количество книг Шукшина или Эренбурга.

Словом, не будем полагаться на удачу и будем искать книгу Маяковского следующим образом. Перейдем к слоту, находящемуся на полке ровно посредине, возьмем находящуюся там книгу и посмотрим, кто ее автор. Предположим, что вы нашли книгу Ефремова. Это не просто не та книга, которую мы ищем; поскольку книги отсортированы по автору, мы точно знаем, что среди книг левее найденной книги Ефремова нужной нам быть не может. Просмотрев только одну книгу, мы исключили из рассмотрения половину книг на полке! Любая книга Маяковского должна быть на правой половине полки. Так что теперь мы найдем слот посредине правой половины полки и посмотрим, какая книга находится в этом месте. Предположим, что это книга Льва Толстого. Далее найдем книгу в слоте посреди третьей четверти полки, в которой только и может располагаться книга Маяковского. Если это Маяковский, мы нашли то, что искали. Если нет, мы снова можем исключить половину оставшихся книг. В конце концов мы либо найдем книгу Маяковского, либо доберемся до такой мелкой части полки, в которой уже не будет ни одного слота, который мог бы содержать искомую книгу. В последнем случае можно заключить, что книги Маяковского на полке нет.

В компьютере мы выполняем бинарный поиск в массиве. В любой момент мы рассматриваем только подмассив, т.е. часть массива между двумя индексами

(включительно). Назовем их p и r. Первоначально $p = 1$ и $r = n$, так что в начале работы подмассив совпадает со всем массивом. Мы многократно делим подмассив пополам, до тех пор, пока не произойдет одно из двух событий: либо мы найдем искомое значение, либо подмассив окажется пустым (т.е. p станет больше, чем r). Именно это многократное деление подмассива пополам и обеспечивает время работы алгоритма, равное $O(\lg n)$.

Рассмотрим работу бинарного поиска немного более подробно. Предположим, что мы ищем значение x в массиве A. На каждом шаге мы рассматриваем только подмассив, начинающийся с злемента $A[p]$ и заканчивающийся элементом $A[r]$. Поскольку нам придется немного поработать с подмассивами, введем для такого подмассива обозначение $A[p.r]$. На каждом шаге мы вычисляем средину q рассматриваемого подмассива, вычисляя среднее p и r и отбрасывая дробную часть, если таковая имеется: $q = \lfloor (p + r)/2 \rfloor$ (здесь мы используем функцию "пол" \lfloor\rfloor для удаления дробной части числа. При реализации этой операции на языках программирования Java, C или C++ можно просто использовать целочисленное деление, при котором дробная часть отбрасывается). Далее выполняется проверка, не равно ли значение элемента $A[q]$ величине x. Если равно, искомый элемент найден, поэтому можно просто вернуть q как индекс элемента массива A, содержащего значение x.

Если же вместо этого выясняется, что $A[q] \neq x$, то можно воспользоваться предположением, что массив A отсортирован. Поскольку $A[q] \neq x$, имеются две возможности: либо $A[q] > x$, либо $A[q] < x$. Сначала рассмотрим случай $A[q] > x$. Так как массив отсортирован, мы знаем, что больше x не только элемент $A[q]$, но и (рассматривая элементы массива как расположенные слева направо) все элементы, расположенные правее $A[q]$. Таким образом, можно исключить из рассмотрения все элементы в позиции q и справа от нее. Поэтому на следующем шаге p не изменяется, а r устанавливается равным $q - 1$.

Если же выясняется, что $A[q] < x$, то мы знаем, что каждый элемент массива слева от q меньше, чем x, и поэтому можно эти элементы не рассматривать. Поэтому на следующем шаге r не изменяется, а p устанавливается равным $q + 1$.

Вот точное описание процедуры бинарного поиска.

Процедура $\text{Binary} - \text{Search}(A, n, x)$

Вход и выход: те же, что и в Linear-Search.

1. Установить p равным 1 , а r равным n.

2. Пока $p \leq r$, выполнять следующие действия.

A. Установить q равным $\lfloor (p + r)/2 \rfloor$.

B. Если $A[q] = x$, вернуть q.

C. В противном случае $(A[q] \neq x$), если $A[q] > x$, установить r равным $q - 1$.

D. В противном случае $(A[q] < x$) установить r равным $q + 1$.

3. Вернуть значение not-found.

Цикл на шаге 2 не обязательно завершается из-за того, что p становится больше, чем r. Он может завершиться на шаге 2 B , если обнаружит, что $A[q]$ равно x, и вернет q как индекс элемента массива A, равного x.

Для того, чтобы показать, что процедура Binary-Search работает корректно, нам нужно просто показать, что если процедура Binary-Search возвращает на шаге 3 значение not-found, то x отсутствует в массиве. Воспользуемся следующим инвариантом цикла.

В начале каждой итерации цикла в шаге 2 , если x находится где-то в массиве A, то это значение находится в одном из элементов подмассива $A[p.r]$.

Вот краткое доказательство корректности с применением инварианта цикла.

Инициализация. Шаг 1 инициализирует индексы p и r значениями 1 и n соответственно, и поэтому инвариант цикла при первом входе в цикл является истинным.

Сохранение. Выше мы доказали, что шаги 2C и 2D корректно обновляют либо p, либо r.

Завершение. Если x отсутствует в массиве, то в конечном итоге процедура доходит до точки, где p и r равны. Когда это происходит, вычисленное на шаге 2 A значение q будет таким же, как p и r. Если шаг 2 C устанавливает значение r равным $q-1$, то в начале следующей итерации r оказывается равным $p-1$, так что p будет больше r. Если же шаг 2D устанавливает значение p равным $q+1$, то в начале следующей итерации p , оказывается равным $r+1$, и p снова будет больше r. В любом случае условие цикла на шаге 2 будет ложным, и цикл завершится. Поскольку $\rho > r$, подмассив $A[p..r]$ пуст, и, таким образом, значение x не может в нем находиться. Рассматривая контрапозицию инварианта цикла (см. с. 33), находим, что если x отсутствует в подмассиве $A[p.r]$, то его нет нигде в массиве A. Таким образом, процедура корректно возвращает значение NOT-FOUND на шаге 3.

Можно также записать бинарный поиск как рекурсивную процедуру

Процедура $Recursive-Binary-Search(A,p,r,x)$

Вход и выход: входные параметры A и x те же, что и у процедуры Linear-Search, также, как и выход. Входные параметры p и r определяют обрабатываемый подмассив $A[p.r]$

1. Если $p > r$, вернуть NOT-FOUND.

2. В противном случае ($p \leq r$) выполнигь следующие действия.

A. Установить $q = \lfloor (p+r)/2 \rfloor$.

B. Если $A[q] = x$, вернуть q.

C. В противном случае ($A[q] \neq x$), если $A[q] > x$, вернуть $Recursive-Binary-Search(A,p,q-1,x)$.

D. В противном случае ($A[q] < x$) вернуть $Recursive-Binary-Search(A,q+1,r,x)$.

Первоначальный вызов имеет вид $Recursive - Binary - Search(A, 1, n, x)$.

Теперь давайте рассмотрим, почему бинарный поиск выполняется в массиве с n элементами за время $O(\lg n)$. Ключевым явыяется наблюдение, что размер $r - p + 1$ рассматриваемого подмассива уменьшается примерно вдвое на каждой итерации цикла (или при каждом рекурсивном вызове в рекурсивной версии, но давайте пока сосредоточимся на итерационной версии Binary-Search). Легко видеть, что если итерации начинаются с подмассива из s элементов, то на следующей итерации размер подмассива будет равен либо $\lfloor s/2 \rfloor$, либо $s/2 - 1$, в зависимости от того, является ли s четным или нечетным числом, и больше или меньше элемент $A[q]$ значения x. Мы уже видели, что, как только размер подмассива опускается до 1, на следующей итерации процедура завершается. Таким образом, вопрос в том, сколько итераций цикла, вдвое уменьшающих рассматриваемый подмассив нужно выполнить, чтобы его исходный размер n уменьшился до 1 . Это то же количество итераций, что и в случае, если бы мы начали с подмассива размером 1 и на каждой итерации удваивали бы его, пока не будет достигнут размер n. Но это просто возведение в степень путем многократного умножения на 2. Другими словами, при каком значении x величина 2^x достигает n ? Если n представляет собой точную степень 2 , то, как мы уже видели на с. 20 , ответом является число $\lg n$. Конечно, n может не быть точной степенью 2 , и в этом случае ответ может отличаться от $\lg n$ не более чем на 1 . Наконец заметим, что каждая итерация цикла требует постоянного количества времени, т.е. время выполнения отдельной итерации не зависит от размера исходного массива n или от размера рассматриваемого подмассива. Давайте воспользуемся асимптотическими обозначениями для того, чтобы отбросить постоянные множители и члены более низкого порядка. (Равно ли количество итераций $\lg n$ или $\lfloor \lg n \rfloor + 1$? Да какая разница!) В результате мы получаем, что время работы бинарного поиска составляет $O(\lg n)$.

Здесь используются O -обозначения, поскольку хотел получить выражение, охватывающее все случаи. В наихудшем случае, когда значение x в массиве отсутствует, мы многократно делим подмассив пополам, пока рассматриваемый подмассив не окажется пустым, и в этом случае время работы равно $\Theta(\lg n)$. В лучшем случае, когда x обнаруживается в первой же итерации цикла, время работы равно $\Theta(1)$. Нет Θ-обозначения, которое охватывало бы все случаи, но выражение $O(\lg n)$ для времени работы бинарного поиска корректно - конечно, если массив предварительно отсортирован.

Превзойти время $\Theta(\lg n)$ в наихудшем случае можно только при более сложной организации данных и определенных свойствах ключей поиска.

Обратим теперь наше внимание на задачу сортировки, в которой требуется так разместить элементы массива - выполнить их перестановку, - чтобы каждый элемент был меньше или равен следующему за ним. Первый алгоритм сортировки, с которым мы познакомимся, - сортировка выбором, на мой взгляд, является самой простой,

потому именно она тут же пришла мне в голову, когда мне надо было разработать алгоритм сортировки массива. Но этот способ сортировки далеко не самый быстрый.

Вот как сортировка выбором будет работать в случае сортировки книг на книжной полке по авторам. Сначала мы проходим по всей полке и находим книгу, автор которой стоит первым по алфавиту, – скажем, Алеся Адамовича (если на полке две или более книг этого автора, выбираем любую из них). Затем мы меняем эту книгу местами с книгой в первом слоте. Теперь в первом слоте находится книга с автором, первым в алфавитном порядке среди всех авторов, книги которых присутствуют на полке. Затем мы вновь проходим по книжной полке слева направо, начиная с книги в слоте 2, и в слотах со второго по слот n ищем книгу, автор которой стоит первым по алфавиту среди просматриваемых книг. Предположим, что это Борис Васильев. Поменяйте эту книгу с книгой в слоте 2, так что теперь слоты 1 и 2 заняты книгами, авторы которых стоят первыми в алфавитном порядке. Затем надо сделать то же самое для слота 3 и т.д. После того как мы поставим нужную книгу в слот $n-1$ (возможно, это кннга Шолохова), сортировка выполнена, так как остается только одна книга (скажем, Шукшина), и она находится в том слоте, где и должна находиться, - в слоте n. Чтобы превратить этот рассказ в компьютерный алгоритм, заменим книжную полку массивом, а книги - его элементами. Вот что получается.

Процедура $Selection-Sort(A,n)$

Вход:

A: сортируемый массив.

n : количество сортируемых элементов в массиве A.

Результат: элементы массива A отсортированы в неубывающем порядке.

1. Для $i=1$ до $n-1$:

A. Присвоить переменной smallest индекс наименьшего элемента в подмассиве $A[i..n]$

B. Обменять $A[i] \leftrightarrow A$[smallest].

Поиск наименьшего элемента в $A[i.n]$ представляет собой вариант линейного поиска. Сначала объявим наименьшим элементом $A[i]$, а затем просканируем остальную часть подмассива, обновляя индекс наименьшего элемента каждый раз, когда находим элемент, меньший, чем текущий наименьший. Вот какой вид имеет уточненная процедура сортировки.

Процедура $Selection-Sort(A,n)$

Вход и результат: те же, что и ранее.

1. Для $i=1$ до $n-1$:

A. Установить значение переменной smalles! равным i.

B. Для $j=i+1$ до n :

i. Если $A[j] < A[$ smallest $]$, присваиваем переменной smallest значение j.

C. Обменять $A[i] \leftrightarrow A[$ smallest $]$.

Эта процедура имеет так называемый «вложенный» цикл: цикл на шаге 1B вложен в цикл на шаге 1. Внутренний цикл выполняет все свои итерации для каждой отдельной итерации внешнего цикла. Обратите внимание, что начальное значение j во внутреннем цикле зависит от текущего значения i во внешнем цикле. Приведенная ниже иллюстрация (рис. 58) показывает, как сортировки выбором работает с массивом из шести элементов.

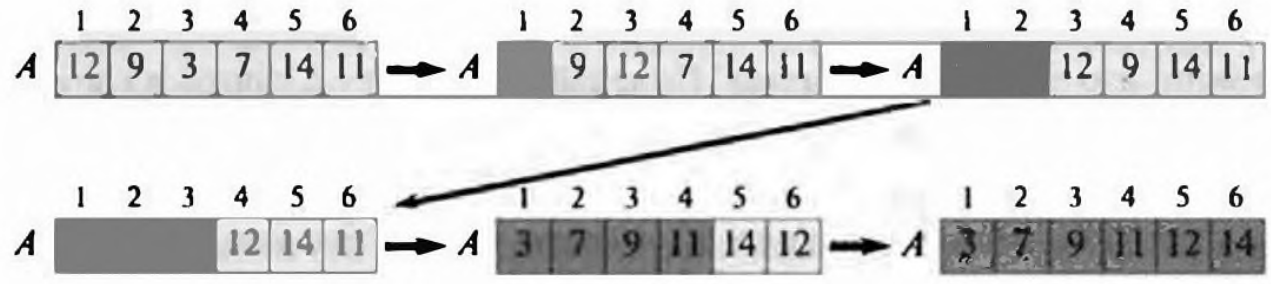


Рисунок 1. Сортировка выбором

Исходный массив показан в верхнем левом углу, и каждое следующее изображение представляет массив после очередной итерации внешнего цикла. Более темным цветом показаны элементы подмассива, о котором точно известно, что он отсортирован.

Если вы хотите воспользоваться инвариантом цикла для доказательства того, что процедура Selection-Sort правильно сортирует массив, вам потребуется по одному инварианту для каждого цикла. Процедура доказательства достаточно проста, так что здесь приведены только инварианты цикла, доказательство остается читателю в качестве небольшого упражнения.

В начале каждой итерации цикла на шаге 1 подмассив $A[1..i-1]$ содержит $i-1$ наименьших элементов массива в отсортированном порядке.

В начале каждой итерации цикла на шаге 1 B элемент $A[$ smallest $]$ представляет собой наименьший элемент в подмассиве $A[i..j-1]$.

Чему же равно время работы процедуры Selectio-Sort? Покажем, что оно равно $\Theta(n^2)$. Ключевым моментом анализа является выяснение количества итераций, выполняемых внутренним циклом, с учетом того, что каждая итерация выполняется за время Θ (1). (Константные множители для верхней и нижней границ в Θ-обозначениях могут быть различны, поскольку в каждой конкретной итерации присваивание значения переменной smallest может как произойти, так и не произойти.) Давайте посчитаем количество итераций с учетом значения переменной внешнего цикла. Когда i равно 1 , внутренний цикл выполняет итерации для j, изменяющегося от 2 до n, или $n-1$ итерацию. Когда i равно 2, внутренний цикл выполняет итерации для j в диапазоне от 3 до n, или $n-2$ итерации. Каждый раз с увеличением переменной внешнего цикла на единицу внутренний цикл выполняется на один раз меньше. В общем случае внутренний цикл выполняется $n-i$ раз. В последней итерации внешнего цикла, когда i равно $n-1$, внутренний цикл

выполняется только один раз. Таким образом, общее количество итераций внутреннего цикла равно $(n-1)+(n-2)+(n-3)+\cdots+2+1$.

Такая сумма известна под названием арифметическая прогрессия, и вот очень важный факт об арифметических прогрессиях: для любого неотрицательного целого k

$$k+(k-1)+(k-2)+\cdots+2+1=\frac{k(k+1)}{2}$$

Подставляя $n-1$ вместо k , мы видим, что общее количество итераций внутреннего цикла равно $(n-1)n/2$, или $(n^2-n)/2$. Воспользуемся асимптотическими обозначениями, чтобы избавиться от членов более низкого порядка $(-n)$ и постоянного множителя $(1/2)$. Теперь можно утверждать, что общее количество итераций внутреннего цикла представляет собой $\Theta(n^2)$. Следовательно, время работы сортировки выбором равно $\Theta(n^2)$. Обратите внимание, что это выражение, охватывающее все случаи. Внутренний цикл выполняется $\Theta(n^2)$ раз независимо от фактических значений элементов массива.

Вот еще один способ увидеть без использования арифметических прогрессий, что время работы равно $\Theta(n^2)$. Покажем отдельно, что время представляет собой как $O(n^2)$, так и $\Omega(n^2)$; объединение асимптотических верхней и нижней границ даст нам $\Theta(n^2)$. Чтобы убедиться, что время работы представляет собой $O(n^2)$, заметим, что в каждой итерации внешнего цикла внутренний цикл выполняется не более $n-1$ раза, что равно $O(n)$, поскольку каждая итерация внутреннего цикла требует постоянного количества времени Поскольку внешний цикл выполняет итерацию $n-1$ раз, что тоже ни что иное, как $O(n)$, общее время, затраченное во внутреннем цикле, представляет собой $O(n)$, умноженное на $O(n)$, или $O(n^2)$. Чтобы убедиться, что общее время работы представляет собой $\Omega(n^2)$, заметим, что в каждой из первых $n/2$ итераций внешнего цикла внутренний цикл выполняет по меньшей мере $n/2$ итераций, т.е. в общей сложности как минимум $n/2$ умноженное на $n/2$ итераций, или $n^2/4$ итераций. Поскольку каждая итерация внутреннего цикла выполняется за постоянное количество времени, можно сделать вывод, что общее время выполнения процедуры представляет собой по меньшей мере константу, умноженную на $n^2/4$, или $\Omega(n^2)$.

Два последних замечания о сортировке выбором. Во-первых, время работы $\Theta(n^2)-$ наихудшее среди всех алгоритмов сортировки, которые мы будем рассматривать. Вовторых, если внимательно изучить работу сортировки выбором, мы увидим, что время работы $\Theta(n^2)$ обусловлено сравнениями на шаге 1Bi. Однако количество перемещений элементов массива равно только $\Theta(n)$, поскольку шаг 1 C выполняется только $n-1$ даз. Если перемещение элементов массива требует большого времени - например, если они велики или располагаются на медленном устройстве типа диска, - то сортировка выбором может оказаться вполне разумным решением.

1.2. Привила написания алгоритма на псевдокоде

Для описания алгоритмов используются интуитивно понятные английские термины [39]. Это сделано для того, чтобы рассматриваемые примеры можно было применять в большинстве языков программирования. Однако часто использование алгоритма связано с некоторыми нюансами, которые приводятся в псевдокоде. Псевдокод – это текст, похожий на язык программирования, но не являющийся таковым. Он описывает структуры и летали, которые вам понадобятся для применения алгоритма в полноценном коде вне зависимости от используемого языка программирования.

В следующем фрагменте показан пример псевдокода для вычисления наибольшего общего делителя (НОД) двух целых чисел.

```
// Находим наибольший общий делитель для a и b.
// GCD(a, b) = GCD(b, a Mod Ь).
Integer: Gcd(Integer: a. Integer: b)
    While (b != 0)
        // Вычисляем остаток.
        Integer: remainder = a Mod b
        // Находим наибольший общий делитель для Ь и остатка,
        a = b
        b = remainder
    End While
        // Наибольший общий делитель для a и 0 - это a.
        Return a
End Gcd
```

Псевдокод начинается с комментария, который предваряется символом //.

В первой строке кода происходит объявление алгоритма. В нашем случае это алгоритм Gcd. который возвращает результат в виде целого числа и принимает два параметра с именами a и Ь, каждый из которых также является целым числом.

Части кода, которые выполняют задачу и по требованию возвращают результат, обычно называются подпрограммами, методами, процедурами, субпроцедурами или функциями.

Далее следует сам метод. Его начинает строка While. открывающая цикл. Код, идущий после этого оператора, будет исполняться до тех пор. пока действует заданное условие.

Цикл заканчивается оператором End While. Использовать последний не обязательно – на конец цикла уже указывает отступ, но все же лучше обозначить блок команд.

О завершении метода свидетельствует оператор Return. Рядом с ним указана величина, которую должен вернуть приведенный алгоритм. Если целью алгоритма является не получение конкретного значения, а, например, систематизация величии или построение структуры данных, то после оператора Return возвращаемая величина не ставится.

Рассмотренный код ничем не отличается от обычного программного. В последующих примерах вы можете встретить описание инструкции либо величины, приведенное на русском языке в угловых скобках (< >). Это значит, что программный код на основе инструкций вам нужно дописать самостоятельно.

После объявления параметра или переменной (в алгоритме Gcd это параметры a и b. а также переменная remainder) задается их тип с обязательным использованием двоеточия. Например, Integer: remainder. Для простых циклических переменных с целыми числами подобной процедурой можно пренебречь, достаточно написать For i = 1 To 10.

Еще одно отличие псевдокода от других языков программирования - возможность использовать цикл For вместе с оператором Step, указывающим на величину. по которой изменяется циклическая переменная. В конце цикла не забудьте написать Next i (где *i* – переменная цикла), чтобы напомнить о его окончании.

Обратите внимание на следующий псевдокод:

```
For 1 - 100 To 0 Step -5
    // Вычисляем-
Next i
```

Он равнозначен нижеприведенному коду C#:

```
for (int i = 100; i >= 0; i -= 5) {
    // Вычисляем-
}
```

В псевдокодах встречается If-Then-Else, Case и некоторые другие выражения. Они являются базовыми для всех языков программирования и должны быть вам знакомы. Остальное, что может понадобиться для кода, записывается в комментариях.

Структура данных, которая может быть вам незнакома (в некоторых языках программирования ее нет), – это List. Она подобна саморасширяющемуся массиву. С помощью метола Add к концу имеющегося списка данных можно добавить еще один элемент. Например, следующий псевдокод создает List Of Integer, в котором содержатся числа от 1 до 10.

```
List Of Integer: numbers
For i = 1 To 10
    numbers.Add(i)
Next i
```

После того как список объявлен, псевдокод может работать с ним. как с обычным массивом, и обращаться к любому его элементу. Однако в отличие от массивов, списки позволяют удалять какой бы то пи было элемент и добавлять его в какую угодно позицию.

Многие алгоритмы написаны в виде методов и функций, возвращающих результат. В этом случае в объявлении функции нужно указать тип получаемых данных. Если функция выполняется без возвращения результата, тип данных не задается.

В следующем псевдокоде содержатся два метода.

```
// Возвращает удвоенное значение введенного числа.
Integer: DoubleIt(Integer: value)
    Return 2 * value
End DoubleIt
// Следующая функция выполняется без возвращения результата.
DoSomething(Integer: values(l)
    // Код.
End DoSomething
```

Первая функция DoubleIt получает и возвращает целое число, удваивая его значение в процессе работы. Вторая функция DoSomething использует в качестве параметра массив целых чисел, называемых величинами. Она также выполняет задание (например, перемешивает или сортирует величины), но результат не возвращает. Обратите внимание и на то, что в книге индексация элементов внутри массива начинается с 0, то есть если массив состоит из трех элементов, индексы у них будут 0,1 и 2.

Пожалуй, единственный существенный недостаток псевдокода – отсутствие компилятора, способного обнаружить ошибки.

СПИСОК ЛИТЕРАТУРЫ

1. Игошин В.И. Математическая логика и теория алгоритмов : учеб, пособие для студ. высш. учеб, заведений / В. И. Игошин. – 2-е изд. стер. – М.: Издательский центр «Академия». 2008. – 448 с.

2. Глухов М. М., Шишков А. Б. Математическая логика. Дискретные функции. Теория алгоритмов: Учебное пособие. – СПб.: Издательство «Лань», 2012. – 416 с.

3. Геут К. Л. Математическая логика и теория алгоритмов : учсб.-метод. пособие / К. Л. Геут, С. С. Титов. – Екатеринбург : УрГУ ПС, 2017. – 85 с.

4. Мендельсон Э. Введение в математическую логику. – М.: Наука, 1976. – 320 с.

5. Зюзьков В. М. Математическая логика и теория алгоритмов : учебное пособие / В. М. Зюзьков. – Томск : Эль Контент, 2015. – 236 с.

6. Успенский В. Л. Труды но нематематике / В. А. Успенский. 2-е изд,, испр. и доп. : В 5 кн.– М. : ОГИ : Фоад «Магматические этюды», 2012. – Кн. 2 : Философия (с приложением трёх писем П. .П. Трауберг к автору). – 2014. – 566 с.

7. Лихтарников Л. М., Сукачева Т. Г. Математическая логика. Курс лекций. Задачник-практикум и решения: Учебное пособие. 4-е изд., стер. – СПб.: Издательство «Лань», 2009. – 288 с.

8. Вялый М., Подольский В., Рубцов А., Шварц Д., Шень А. Лекции по дискретной математике. 448 с. https://publications.hse.ru/mirror/pubs/share/direct/393719078.pdf

9. Пальчунов, Д. Е. Математическая логика и теория алгоритмов : учеб, пособие / Д. Е. Пальчунов ; Новосиб. гос. ун-т. - Новосибирск, 2016. – Ч. 1. – 96 с.

10. Барашев В.П.. Унучек С.А. Дискретная математика : Учебное пособие Федеральное государственное бюджетное образовательное учреждение высшего профессионального образования «Московский государственный технический университет радиотехники, электроники и автоматики». – М., 2012. –268 с., электронное издание.

11. Чашкин А. В. Дискретная математика : учебник для учреждений высш. проф. образования / А. В. Чашкин. – М.: Издательский центр «Академия», 2012. – 352 с.

12. Гринченков Д.В. Математическая логика и теория алгоритмов для программистов : учебное пособие / Д.В. Гринченков, С.И. Потоцкий. – М.: КНОРУС, 2010. – 208 с.

13. Вайнштейн К. В. Математическая логика и теория алгоритмов : учеб, пособие / Ю. В. Вайнштейн, Т. Г. Пенькова, В. И. Вайнштейн. – Красноярск : Сиб. федер. ун-т, 2019. – 110 с.

14. Новиков Ф. А. Дискретная математика для программистов. Учебник для вузов. 3-е изд. – СПб.: Питер, 2009. – 384 с.

15. Марченков С. С. Основы теории булевых функций. М.: ФИЗМАТЛИТ, 2014. – 136 с.

16. Гаврилов Г. П., Сапоженко А. А. Задачи и упражнения по дискретной математике: Учеб, пособие. – 3-е изд., перераб. – М.: ФИЗМАТЛИТ, 2005. – 416 с.

17. Кормен Т. Х., Лензерсон, Ч. И., Ривест Р. Л., Штайн К. Алгоритмы: построение и анализ. 2-е издание : Пер. с англ. – М. : Издательский дом «Вильямс», 2011. – 1296 с.

18. Алексеев В.В. Теория алгоритмов. Учебно-методическое пособие. СарФТИ НИЯУ МИФИ, 2021. – 100 с.

19. https://education.yandex.ru/handbook/algorithms

20. Ахо А. В., Хопкрофт Д. Э., Ульман Д. Д. Структуры данных и алгоритмы : Пер. с англ. : М. : Издательский дом «Вильямс», 2003. – 384 с.

21. Граничин О.Н. Рандомизированные алгоритмы в задачах обработки данных и принятия решений. Системное программирование. Санкт-Петербургский государственный университет, Т. 6, № 1. С. 141–162.

22. Регеда В. В., Регеда О. Н. Основы алгоритмизации. Методические указания по курсу «Информатика». Пенза: Издательство Пензенского государственного университета, 2008. – 56 с.

23. Туаева М. Т. Алгоритмы. Основы алгоритмизации: Учебное пособие. – Владикавказ: Владикавказский торгово-экономический техникум, 2020. – 51 с.

24. Алгоритмы. Основные алгоритмические конструкции [Текст]: сб. задач / сост. С.А. Рогозин. – Челябинск: Изд-во Челяб. гос. пед. ун-та, 2008. 42 с.

25. Глухова Л.А. Основы алгоритмизации и программирования : лаб. практикум для студ. спец. I-40 01 01 «Программное обеспечение информационных технологий» дневн. формы обуч. В 4 ч. Ч. 3 / Л.А. Глухова, Е П. Фадеева, Е.Е. Фадеева. – Минск : БГУИР, 2007. – 52 с.

26. Макконнелл Дж. Основы современных алгоритмов. 2-е дополненное издание. Москва: Техносфера, 2004. – 368с.

27. Орлов С. А. Теория и практика языков программирования: Учебник для вузов. Стандарт 3-го поколения. – СПб.: Питер. 2013. – 688 с.

28. Анисимов А.Е. Ведение в алгоритмизацию задач / А.Е. Анисимов. – Ижевск: Изд-во «Удмуртский университет», 2017. – 44 с.

29. Конова Е. А. Алгоритмы и программы. Язык С++ : учебное пособие для СПО / Е. А. Конова, Г. А. Поллак. – 3-е изд., стер. – Санкт-Петербург : Лань, 2022. – 384 с.

30. Паклина В.М., Паклина Е.М. Основы алгоритмизации и программирования. – Екатеринбург: ГОУ ВПО УГТУ−УПИ, 2009. – 91 с.

31. Ворожцов А.В., Винокуров Н.А. Лекции Алгоритмы: Построение, анализ и реализация на языке программирования Си. – Москва: МФТИ, 2007. – 452 с.

32. Мусатов Д.В. Сложность вычислений : конспект лекций. – Москва: МФТИ, 2017. – 224 с.

33. Седжвик Р. Фундаментальные алгоритмы на С++. Анализ / Структуры данных / Сортировка / Поиск : Пер. с англ. / Роберт Седжвик. – К.: Издательство «ДиаСофт», 2001. – 688 с.

34. Имран А. 40 алгоритмов, которые должен знать каждый программист на Python. – СПб.: Питер. 2023. – 368 с.

35. Hetland M. L. Python Algorithms: Mastering Basic Algorithms in the Python Language. – 2010. – 337 p.

36. Пинчук В.П. Анализ и построение алгоритмов. Краткий конспект лекций для спец. КСС. – Запорожье: ЗНТУ, 2012.

37. Филиповский В.М. Основы программирования и алгоритмизации. Ч. 2. Практикум по алгоритмизации: Учебное пособие. – СПб.: СПбПУ. 2022. – 71 с.

38. Кормен Т. Х. Алгоритмы: вводный курс. : Пер. с англ. – М. : ООО «И.Д. Вильямс», 2014. – 208 с.

39. Стивенс Р. Алгоритмы. Теория и практическое применение / Род Стивене. – Москва : Издательство «Э». 2016. – 544 с.